TECHNICAL SPECIFICATIONS (TECH_SPEC)

Arsitektur Teknis, Sistem 3D, Multiplayer P2P WebRTC, dan Optimasi Performa

Ø=Þàþ Technical Specifications (TECH_SPEC): Adabku

Arsitektur Teknis, Sistem 3D, Multiplayer P2P, dan Optimasi Performa

1. Arsitektur Sistem & Tech Stack

Adabku dirancang sebagai aplikasi web berbasis *Single Page Application* (SPA) dengan pemisahan tegas antara logika rendering 3D grafis tinggi dan lapisan antarmuka pengguna (UI overlay).

```
% % % %
```

proyektil laser, detektor orb kuis, serta transmiter multiplayer tanpa memicu re-render DOM React.

2.2 State Reaktif Tingkat Tinggi (`src/store/gameStore.js`)

Zustand hanya digunakan untuk mengelola transisi langkah UI, status kuis, skor, amunisi air, dan sinkronisasi status P2P:

- `phase`: `'start'` | `'playing'` | `'paused'` | `'quiz'` | `'puzzle'` | `'finished'`
- `onboardingStep`: `1` (Landing) | `2` (Karakter) | `3` (Mode) | `4` (Pilih Level)
- `connectionStatus`: `'connecting'` | `'waiting'` | `'connected'`
- `unlockedLevels`: Array level yang terbuka (disimpan di `localStorage` kunci `adabku:unlocked_levels`).

3. Sistem Kontrol & Kamera First-Person 360°

Komponen `FirstPersonPlayer.jsx` menggabungkan kontrol pergerakan pemain dan kendali pandangan bebas 360°:

3.1 Kontrol Input Multi-Perangkat

1. **Desktop/Laptop:**
- Tombol `W` / `A` / `S` / `D` atau Tombol Panah: Pergerakan maju, mundur, menyamping.
- `Space`: Melompat (*Jump Velocity* = 5.2 m/s, gravitasi = 18 m/s²).
- `Ctrl` / `Shift`: Menunduk (*Crouch* squash = 0.95m eye height).
- Tombol `F`: Menembakkan Pistol Air (Level 5).
- Pointer Drag: Menyeret kursor mouse untuk memutar sudut pandang yaw & pitch.

2. **Mobile/Tablet (Smartphone):**
- **Virtual Joystick Kiri::** Mengontrol vektor pergerakan `(normX, normY)`.
- **Tombol Aksi Kanan::** `+ LOMPAT`, `+ MENUNDUK`, dan `Ø=Ü¦ TEMBAK`.
- **Touch Drag 360° (`onTouchMove`)::** Mengusapkan jari di area layar bebas untuk memutar sudut kamera 360° dengan sensitivitas `0.0042 rad/px`.

3.2 Vektor Arah 360°

Arah pergerakan dihitung secara dinamis relatif terhadap arah pandang kamera:

```
const fwdX = Math.sin(look.yaw)
const fwdZ = -Math.cos(look.yaw)
const rightX = -fwdZ
const rightZ = fwdX

const moveWorldX = fwdX * inputFwd + rightX * inputRight
const moveWorldZ = fwdZ * inputFwd + rightZ * inputRight
```

4. Arsitektur Multiplayer P2P (PeerJS & STUN)

Modul `src/services/multiplayer.js` mengimplementasikan jaringan WebRTC P2P yang dapat bekerja lintas perangkat:

```
[ Laptop / Client A ]  <====== STUN Server Google ======>  [ HP / Client B ]
        %                  (stun.l.google.com:19302)               %
        %                                                          %
        % % % % % % % % % % % % % % % %  Direct WebRTC DataChannel % % % % % % % % % % % % % % % %
                              (Room: ADABKU123)
                            Broadcast @ 20 FPS (50ms)
```

4.1 Konfigurasi ICE & STUN

```
const PEER_CONFIG = {
  debug: 0,
  config: {
    iceServers: [
      { urls: 'stun:stun.l.google.com:19302' },
      { urls: 'stun:stun1.l.google.com:19302' },
    ],
  },
}
```

4.2 Alur Host-Client Deterministik

1. **Percobaan Host:** Perangkat pertama mendaftarkan ID Host `adabku-room-[KODEROOM]`. Jika berhasil, status berubah menjadi `waiting` (Menunggu Pemain).

2. **Fallback Client:** Jika ID Host sudah terdaftar (`err.type === 'unavailable-id'`), perangkat secara otomatis mendaftar sebagai Client dengan ID acak unik dan menghubungi `adabku-room-[KODEROOM]`.

3. **Dual-Channel Fallback:** Menggunakan `BroadcastChannel` lokal untuk pengujian instan antar tab pada perangkat

yang sama tanpa bergantung pada broker eksternal.

4. **Sinkronisasi Posisi (50ms):**

```
const packet = {
  type: 'update_pos',
  id: myPeerId,
  name: playerName,
  gender: playerGender,
  position: [player.x, player.y, player.z],
  rotation: player.yaw,
  level: currentLevel,
  crouchT: player.crouchT,
  isJumping: player.y > 0.35,
}
```

5. **Render 3D Nametag Native:** Menggunakan Three.js `CanvasTexture` + `Billboard` yang sangat efisien dan bebas dari masalah *memory leak* atau *CDP timeout*.

---

5. Level 5: Arena Terbuka, Menara Laser, & Dinding Pembatas

Level 5 memindahkan pemain dari koridor lurus menuju plaza melingkar terbuka (*Circular Plaza*) berdiameter 50 meter:

5.1 Dinding Pembatas Tak Terlihat (*Invisible Wall Boundaries*)

Pemain dicegah keluar dari lapangan menggunakan kalkulasi jarak radial di dalam *loop* fisika:

```
const distToCenter = Math.hypot(nextX, nextZ)
const ARENA_RADIUS = 23.8 // Batas luar lapangan
if (distToCenter > ARENA_RADIUS) {
  nextX = (nextX / distToCenter) * ARENA_RADIUS
  nextZ = (nextZ / distToCenter) * ARENA_RADIUS
}

const TOWER_RADIUS = 3.8 // Batas pilar menara tengah
if (distToCenter < TOWER_RADIUS) {
  nextX = (nextX / distToCenter) * TOWER_RADIUS
  nextZ = (nextZ / distToCenter) * TOWER_RADIUS
}
```

5.2 Mekanisme Pistol Air & Proyektil Pool

- Komponen `WaterProjectilesHandler` menggunakan sistem *Object Pooling* (8 slot proyektil) tanpa alokasi memori dinamis di dalam `useFrame`.
- Proyektil meluncur menuju pusat Sumber Laser `(0, 4.2, 0)` dengan kecepatan 25 m/s.
- Menabrak inti menara memicu `damageTower()` yang mengurangi HP Sumber Laser (dari 8 s.d. 0) dan memadamkan perilaku buruk untuk meraih kemenangan mutlak.

---

6. Optimasi Build & Solusi Memori Terbatas

Pada mesin dengan kapasitas RAM terbatas, compiler Rolldown Vite 8 dapat mengalami kehabisan memori jika menjalankan *multithreaded transform*.

6.1 Konfigurasi Single-Thread Rust (`vite.config.js`)

```
process.env.RAYON_NUM_THREADS = process.env.RAYON_NUM_THREADS || '1'
```

Penetapan variabel lingkungan ini secara otomatis menstabilkan build menjadi alokasi memori linear rendah tanpa mengorbankan kecepatan (proses build selesai dalam 1.08 detik).

6.2 Chunk Splitting Manual

Bundle produksi dipisah menjadi beberapa berkas vendor mandiri:

- `three-vendor.js`: Core Three.js library
- `r3f-vendor.js`: React Three Fiber & Drei helpers
- `peerjs-vendor.js`: WebRTC PeerJS engine
- `react-vendor.js`: React 19 & React DOM runtime